

GraphWorld 内部文档

目标：统一 Object、State、Edge、Action、Rule、Process 和 Task 的定义，并明确从任务定义到可解任务实例的自动生成链路。

成熟度：Implemented 代码已实现；Proposed 目标确定但未实现；Open 仍需决策。

1. 文档目标与核心决策

GraphWorld 将世界表示为随动作和时间变化的图。任务只描述目标，不预先规定动作序列；规划器根据初始图、目标和世界变化规则寻找计划。

当前规则：

- 对象模板声明类型、状态、能力和放置约束；对象实例是场景图中的节点。
- Edge 表示位置、承载、持有、连接和控制关系；State 表示节点自身属性。
- Action 是主体主动执行的原子接口；Rule 是动作触发的即时效果；Process 是过程变化。
- 原子任务由初始状态和一个未满足的目标状态组成；组合任务是多个目标状态的合取。
- 六类任务是语义分类，只用于统计分析，不参与规划语义。
- 新物体和新玩法优先通过对象能力、领域规则、过程和配方扩展，不增加一类专用原子动作。
- has_water 使用 0-100 数值资源。0 表示无水，100 表示装满；消耗过程可以产生中间值。
- 任务实例只有在规划成功且计划能够在 runtime 重放后，才计入已验证任务集。

2. 世界建模：Object / State / Edge

设时刻 t 的世界图为：

$$G_t = (V_t, E_t, S_t, W_t)$$

其中 V_t 是节点集合， E_t 是有类型的关系边， S_t 是节点状态， W_t 是时间、天气和房间环境等全局状态。GraphWorld ontology 是这些节点类型、状态、关系和变化规则的统一词汇与约束。

2.1 Object

对象分为模板和实例：

```
ObjectTemplate = semantic_type + family + node_type + capabilities
                + default_states + placement_spec + required_systems

ObjectInstance = object_id + template_reference + current_states + current_edges
```

字段	含义	说明
semantic_type	语义类型	物体在世界中的具体类别，例如 vase、sink、washing_machine
family	功能类别	更高层的物体分组，例如容器、食品、工具、家具、设备
node_type	节点类型	图结构中的节点角色，例如可移动物体、不可移动物体、控制器
capabilities	能力集合	物体支持的通用能力，例如可抓取、可放置、可打开、可清洁、可计时运行
default_states	默认状态	新实例创建时的初始状态，例如 is_open=False、has_water=0
placement_spec	放置约束	物体允许出现的房间、父节点、表面或容器，以及容量要求
required_systems	依赖系统	物体正常运行所需的系统，例如时间、温度、烹饪或资源消耗系统
object_id	对象唯一标识	场景中具体实例的 ID，例如 vase_01
template_reference	对象模板引用	该实例所使用的 ObjectTemplate，决定其类型、能力和合法状态
current_states	当前状态	实例在当前时刻的状态值，会随 Action、Rule 和 Process 改变
current_edges	当前关系边	实例当前的位置、持有、组成和控制关系，例如 in、on、held_by

atomic/composite 与 movable/fixed 是不同维度：

- 原子物体是在当前粒度下不再拆分的实体，例如盘子、电线、纸巾盒。
- 复合物体由多个实体及组成关系构成，例如制作汉堡、焊接组件、插花花瓶。
- 可移动物体能够成为 `pick` 的对象；不可移动物体作为房间结构、设备或放置目标。
- 数量资源保存在资源节点的数值状态中，例如 `tissuebox.count=100`，不创建 100 个纸巾节点。

物体库完整清单见附录 A。

2.2 State

State 是节点自身的有类型属性。

状态组	状态	值域	作用	成熟度
控制	<code>is_open</code> 、 <code>is_on</code> 、 <code>is_pressed</code> 、 <code>is_running</code>	Boolean	门、容器、控制器和设备运行状态	Implemented
一般条件	<code>is_dirty</code> 、 <code>is_wet</code> 、 <code>is_broken</code> 、 <code>is_blocked</code> 、 <code>folded</code>	Boolean	清洁、干湿、完整性、阻塞和构型条件	Implemented
数量资源	<code>cycle_remaining</code> 、 <code>fill_level</code> 、 <code>uses_left</code> 、 <code>count</code> 、 <code>amount</code> 、 <code>capacity</code>	Number	计时、液体、使用次数、库存和容量	Implemented
水资源	<code>has_water</code>	Number <code>[0,100]</code>	水槽和容器当前水量	Proposed：当前代码为 Boolean
材料与热状态	<code>temperature</code> 、 <code>is_cooked</code> 、 <code>is_burnt</code> 、 <code>is_frozen</code> 、 <code>is_boiling</code>	Number/Enum/Boolean	加热、冷却和加工结果	Partial：状态已注册，过程未完整覆盖

状态组	状态	值域	作用	成熟度
生命周期	<code>is_rotten</code> 、 <code>is_wilted</code> 、 <code>vitality</code>	Boolean/Number	食物腐败和植物生命状态	Implemented
派生状态	<code>is_full</code> 等	Boolean	由数量阈值推导，避免与源数值独立更新	Partial

状态更新必须记录 `old_value -> new_value`。同一语义只保留一个权威状态，例如清洁统一使用 `is_dirty=false`，不再独立维护 `is_clean=true`。

2.3 Edge

核心关系保留为最小、定向的集合：

关系	源节点	目标节点	语义	主要修改者	成熟度
<code>at</code>	主体	房间或固定物体	主体所在位置	<code>move</code>	Implemented
<code>in</code>	物体	容器或房间	物体位于内部	<code>place</code> 、Rule、Process	Implemented
<code>on</code>	物体	表面	物体位于表面	<code>place</code> 、Rule、Process	Implemented
<code>held_by</code>	物体	主体	物体被持有	<code>pick</code> 、 <code>place</code>	Implemented
<code>near</code>	主体或物体	物体或设备	满足局部交互距离	<code>move</code> 、 <code>place</code>	Implemented
<code>connected</code>	房间	房间	可导航拓扑连接	场景生成器	Implemented
<code>controls</code>	控制器	设备或作用区域	控制关系	场景生成器	Implemented
<code>part_of</code>	部件	复合物体	组成关系	配方或装配 Rule	Proposed

`contains` 是 `in` 的反向查询，不作为独立权威边存储。`inside`、`inside_room`、`ontop`、`neighbour` 等历史写法在读入时归一化到核心关系。

每个可移动物体同一时刻只保留一个位置父节点。执行 `pick` 或 `place` 时先移除旧位置边，再添加新边。

3. 世界变化机制：Action / Rule / Process

世界变化统一表示为：

```
TransitionEffect = state_updates + resource_updates
                  + add_edges + remove_edges + spawn + despawn
```

字段	含义	说明
<code>state_updates</code>	状态更新	修改节点的一般状态，例如 <code>is_dirty: true -> false</code> 、 <code>is_on: false -> true</code>
<code>resource_updates</code>	资源更新	修改可计数或可消耗资源，例如 <code>count: 100 -> 99</code> 、 <code>has_water: 100 -> 80</code>
<code>add_edges</code>	新增关系边	建立新的图关系，例如放下盘子后新增 <code>in(plate, cabinet)</code>

字段	含义	说明
<code>remove_edges</code>	删除关系边	移除失效的图关系，例如抓取盘子时删除 <code>on(plate,table)</code>
<code>spawn</code>	生成节点	在 <code>runtime</code> 中创建新对象实例，例如配方完成后生成汉堡
<code>despawn</code>	删除节点	从活动世界中移除已消耗或销毁的对象，例如制作后移除被消耗的原料

`resource_updates` 在数据结构上也属于状态更新，但单独列出便于检查容量、消耗和补充是否守恒。

3.1 Action

Action 是主体可主动选择的最小接口。动作名不绑定对象；参数化后形成实例化动作，例如 `pick(robot_01, vase_01)`。

当前已实现 10 个动作：

Action	参数	前置条件	效果	成熟度
<code>move</code>	<code>actor,target</code>	目标可达，结构门不阻塞	替换主体的 <code>at/near</code> edge	Implemented
<code>pick</code>	<code>actor,object</code>	物体可移动、同房间、手为空、父容器可访问	<code>object --held_by--> actor</code>	Implemented
<code>place</code>	<code>actor,object,target</code>	主体持有物体，目标可放置、可访问且容量允许	建立 <code>in/on</code> edge	Implemented
<code>open</code>	<code>actor,target</code>	目标支持打开且当前关闭	<code>is_open=true</code> ；水龙头使用 <code>is_on=true</code>	Implemented
<code>close</code>	<code>actor,target</code>	目标支持关闭且当前打开	<code>is_open=false</code> ；水龙头使用 <code>is_on=false</code>	Implemented
<code>press</code>	<code>actor,target</code>	目标可按压，设备输入和门状态合法	切换控制状态或启动过程	Implemented
<code>brush</code>	<code>actor,target</code>	目标可清洁且当前脏	<code>is_dirty=false</code>	Implemented
<code>fold</code>	<code>actor,target</code>	目标是干燥布类	<code>folded=true</code>	Implemented
<code>dump</code>	<code>actor,target</code>	主体持有可倾倒容器，接收目标兼容	清空资源或迁移内容物	Implemented
<code>refill</code>	<code>actor,target,supply</code>	补充物兼容且与主体同房间	资源恢复到容量，消耗补充物	Implemented

以下动作只在不可由现有动作和领域规则表达的直接主体操作时增加：

Candidate Action	用途	成熟度
<code>connect(actor,part_a,part_b)</code>	直接连接、焊接或装配两个兼容部件	Proposed
<code>rotate(actor,object,operation)</code>	魔方转面、旋转零件或改变离散构型	Proposed
<code>consume(actor,resource,target)</code>	主体主动消耗有明确目标的资源	Open

`fill` 不是自由动作。容器装水由位置条件和水龙头操作触发。`discard` 也不需要独立动作；将对象放入垃圾桶或执行 `dump` 后，由领域规则更新其生命周期。

3.2 Rule

Rule 是动作执行后立即触发的领域效果：

```
Rule = trigger_action + conditions + immediate_effects
```

字段	含义	说明
<code>trigger_action</code>	触发动作	启动规则检查的主体动作，例如 <code>open(faucet)</code> 、 <code>place(vase,sink)</code>
<code>conditions</code>	生效条件	规则必须满足的图关系、状态、类型和能力约束
<code>immediate_effects</code>	即时效果	动作完成后立即提交的 <code>TransitionEffect</code> ，不经过时间倒计时

目标规则包括：

Rule	条件	即时效果	成熟度
水槽供水	<code>controls(faucet,sink)</code> 且 <code>open(faucet)</code>	<code>sink.has_water=100</code>	Proposed：当前代码为 Boolean
水槽停水	<code>controls(faucet,sink)</code> 且 <code>close(faucet)</code>	<code>sink.has_water=0</code>	Proposed：当前代码为 Boolean
容器装水	容器 <code>in sink</code> 且 <code>sink.has_water>0</code>	容器 <code>has_water=100</code>	Proposed：当前 Boolean 规则部分支持
布类浸湿	布类 <code>in sink</code> 且 <code>sink.has_water>0</code>	<code>is_wet=true</code>	Partial：当前只在放入水槽时触发
垃圾处置	对象进入兼容垃圾桶或垃圾站	更新位置和处置状态	Partial
控制传播	<code>controls(button,device)</code> 且 <code>press(button)</code>	切换设备或启动过程	Implemented

Rule 不由规划器作为机器人动作选择，但其效果必须进入规划模型，否则 PDDL 计划与 runtime 会产生语义偏差。

3.3 Process

Process 表示跨 tick 的环境变化：

```
ProcessInstance = process_type + participants + remaining
                  + invariants + finish_effects
```

字段	含义	说明
<code>process_type</code>	过程类型	过程采用的通用规则，例如洗衣、烘干、打印、配方制作或自然衰减
<code>participants</code>	参与对象	本次过程涉及的设备、输入物、输出物和作用对象的实例 ID
<code>remaining</code>	剩余时间	距离过程完成还需推进的 tick 数；每次时间推进后递减
<code>invariants</code>	持续约束	过程运行期间必须持续满足的条件，例如设备保持关闭、输入仍在设备内
<code>finish_effects</code>	完成效果	<code>remaining=0</code> 时一次性提交的 <code>TransitionEffect</code> ，例如洗净衣物或生成产物

Process	启动条件	tick/完成效果	成熟度
洗衣	衣物在关闭的洗衣机内并启动	倒计时结束后 <code>is_dirty=false,is_wet=true,folded=false</code>	Implemented
烘干/晾干	湿衣物在烘干机或晾衣架上	倒计时结束后 <code>is_wet=false</code>	Implemented
打印	打印机有纸和墨并启动	消耗资源并生成 <code>receipt</code>	Partial
咖啡制作	咖啡机有水、咖啡豆和杯子并启动	消耗咖啡豆并生成 <code>coffee</code>	Partial
花瓶耗水	花在有水花瓶中	<code>has_water</code> 随时间下降，耗尽后植物活力下降	Proposed：当前仅有布尔耗尽逻辑
食物腐败	易腐食物随时间变化	达到阈值后 <code>is_rotten=true</code>	Implemented

过程完成效果可以同时改变多个状态和边。过程不是原子任务，也不是主体动作。

4. 任务模型：Atomic Goal / Atomic Task / Composite Task

4.1 Atomic Goal

Atomic Goal 是相对于当前版本 ontology 的一个规范化目标文字。原子性是语法边界，不是物理世界中的绝对不可分性。

目标类型	形式	示例
状态目标	<code>state(object,key)=value</code>	<code>plate.is_dirty=false</code>
关系目标	<code>relation(source,type,target)</code>	<code>in(plate,cabinet)</code>

目标类型	形式	示例
存在性目标	<code>exists(object)=value</code>	<code>exists(hamburger_01)=true</code>

一个存在性目标可以由配方过程产生多个内部效果。`exists(hamburger_01)=true` 仍是一个 Atomic Goal，但配方必须同时保证原料消耗、组成关系和输出位置一致。

4.2 Atomic Task

原子任务由初始约束 C_0 和一个尚未满足的 Atomic Goal g 构成：

$$T = (C_0, g) \wedge G_0 \wedge \neg g$$

$G_0 \wedge \neg g$ 排除零步任务。Atomic Task 不等于 Action；完成一个原子任务可能需要多个动作。

```
Atomic Task: in(plate_01, cabinet_01)
Possible Plan: open(cabinet_01) -> pick(plate_01)
               -> move(cabinet_01) -> place(plate_01, cabinet_01)
```

4.3 Composite Task

组合任务共享同一个初始约束，并要求多个 Atomic Goal 同时成立：

$$T = (C_0, \bigwedge_{i=1}^n g_i)$$

例如“把脏盘子洗净、晾干并放回橱柜”的目标是：

```
plate.is_dirty=false
AND plate.is_wet=false
AND in(plate, cabinet)
```

任务只保存目标和必要约束，不保存路线、动作顺序或计划长度。规划器负责复用共享步骤、处理前置条件并决定顺序。

5. 六类任务标签

六类标签根据目标变化的语义计算，用于数据统计、任务采样、难度分层和结果分析。不是互斥分区，不参与 PDDL 求解。

标签	触发依据	典型 Atomic Goal
清洁 <code>clean</code>	洁净或污染状态变化	<code>is_dirty=false</code>
制作 <code>make</code>	目标实体生成，或目标复合结构成立	<code>exists(hamburger)=true</code> 、 <code>part_of(wire, board)</code>
迁移 <code>relocate</code>	非主体物体的位置、容器、表面或持有关系变化	<code>in(plate, cabinet)</code>
操作 <code>operate</code>	设备控制、资源量或构型状态变化	<code>is_on=true</code> 、 <code>has_water=100</code> 、 <code>configuration=solved</code>
交互 <code>interact</code>	主体之间的信息、社会关系或交接结果变化	<code>acknowledged(human, robot)</code>
移动 <code>navigate</code>	主体自身的位置关系变化	<code>at(robot, kitchen)</code>

标签由目标集合计算，不是人工写唯一类别：

```
labels(T) = union(label(g) for g in goals(T))
```

例如，制作汉堡并放入冰箱标记为 `make + relocate`。把物品交给人可以标记为 `relocate + interact`。当移动只服务其他目标时，`move` 是支撑动作；只有主体位置本身属于目标时，任务才带 `navigate` 标签。

6. 任务自动生成：Schema / Grounding / Planning / Replay

自动任务生成分为四层：

层	输入	输出	职责
Schema	ontology + 任务种类	参数查询、初始约束、目标表达式	定义可生成什么任务，不绑定实例 ID
Grounding	schema + scene graph	<code>task_instance.json</code>	绑定对象，检查类型、能力、资源和静态拓扑
Planning	task instance + transition model	PDDL problem + plan	搜索满足目标的动作序列
Replay	plan + runtime	execution trace + final graph	逐步执行计划，验证实际效果与规划语义一致

任务 schema 只声明 `binding`、`initial_constraints`、`requirements` 和 `goals`：

```
{
  "task_id": "store_object",
  "binding": {
    "object": {"query": {"capability": "pickable"}},
    "destination": {"query": {"capability": "place_target"}}
  },
  "initial_constraints": [
    {"not_relation": ["object", "in", "destination"]}
  ],
  "requirements": {
    "world_rules": ["placement_compatibility"]
  },
  "goals": [
    {"relation": ["object", "in", "destination"]}
  ]
}
```

完整流程为：

```
选择 schema
-> 查询并绑定场景对象
-> 检查 C0、类型、能力、资源和静态可达性
-> 生成一个或多个 Atomic Goal
-> 编译 PDDL domain/problem
-> 调用 Fast Downward
-> 在 runtime 重放计划
-> 保存 solved + replay-pass 实例
-> 记录 rejected / unsolved / timeout / replay-failed 反例
```

Grounding 只排除类型不兼容、资源不存在和静态不可达等确定失败。复杂可解性由规划器判断。经典 PDDL 的对象集合固定，因此配方输出在 problem 中预先声明为 `exists=false` 的潜在对象，过程完成后将其切换为 `exists=true`。runtime 可以直接 `spawn` 新节点。

任务覆盖统计要保存：

```
schema_id, instance_id, ontology_version, scene_id, seed,
grounding_status, planner_status, plan_length, replay_status,
initial_graph, goal_literals, plan, final_graph_delta
```

7. 典型任务的完整建模案例

7.1 插花、装水并放回

成熟度：目标语义案例。数值水资源和“开水时作用于水槽已有内容物”的规则为 `Proposed`。

初始图：

```
at(robot, living_room)
on(flower, table)
on(vase, vase_home)
```

```
vase.has_water=0
controls(faucet,sink)
connected(living_room,bathroom)
```

目标:

```
in(flower,vase)
AND vase.has_water=100
AND on(vase,vase_home)
```

一种计划:

```
pick(flower)
-> place(flower,vase)
-> pick(vase)
-> move(bathroom)
-> place(vase,sink)
-> open(faucet)
-> close(faucet)
-> pick(vase)
-> move(living_room)
-> place(vase,vase_home)
```

关键即时规则:

```
open(faucet) -> sink.has_water: 0 -> 100
in(vase,sink) AND sink.has_water>0 -> vase.has_water: 0 -> 100
close(faucet) -> sink.has_water: 100 -> 0
```

花瓶水不会因关闭水龙头而清空。后续自然过程逐步消耗 `vase.has_water`；水耗尽后，植物 `vitality` 才开始下降。该组合任务的标签为 `relocate + operate`。

7.2 制作汉堡

成熟度：目标语义案例。汉堡对象模板、工作台和通用 Recipe Registry 为 `Proposed`。

配方:

```
recipe_id: hamburger
processor: assembly_table
inputs:
  bread: 2
  cooked_patty: 1
  lettuce: 1
duration: 3
consumes: [bread, cooked_patty, lettuce]
output: hamburger
```

目标:

```
exists(hamburger_01)=true
```

机器人只执行通用动作：收集输入，将输入放入工作台并启动设备。Recipe Process 校验输入后倒计时，完成时消耗原料、激活输出节点、建立输出位置和 `part_of` 关系。PDDL 编译器可以生成内部 `finish_recipe` 算子，但该算子不是机器人 Action，也不需要新增 `make_hamburger`。

8. 当前实现覆盖

能力	当前数量或范围	成熟度	事实来源
对象模板	130	Implemented	backend/core/assets/object_library.py
对象语义类型	18 个 ObjectFamily，部分仍由规则推断	Partial	backend/core/assets/object_model.py
注册状态	25	Implemented	backend/core/states.py
Runtime Action	10	Implemented	backend/core/actions.py、 action_schemas.py
Core Task Skill	12	Implemented	backend/core/assets/task_library.py
空间、容器与控制关系	核心关系已注册，仍有历史别名	Partial	backend/core/edges.py
洗衣、烘干和自然衰减	tick transition	Implemented	backend/core/timed_transitions.py
打印和咖啡配方	2 类数据驱动过程	Partial	backend/core/processes.py
has_water: 0-100	数值水资源	Proposed	当前 states.py 和 effects.py 仍按 Boolean 处理
通用 recipe registry	任意输入、设备、输出和完成效果	Proposed	当前仅打印/咖啡专用分支
通用 Schema Grounding	任意任务 schema 到实例	Proposed	尚无统一实例化器
通用 PDDL 编译	scene + schema + rules/processes 到 PDDL	Proposed	当前只有 laundry 专用脚本
PDDL 已验证任务族	1: laundry, 已知计划长度 17	Implemented	backend/tools/try_laundry_fastdownward.py
Planner 到 runtime Replay	自动逐步重放与终态核验	Proposed	尚无通用闭环
六类标签自动计算	goal literals 到多标签	Proposed	当前只有文档 taxonomy

任务数量按实例统计。只有 planner_status=solved 且 replay_status=pass 的任务实例进入已验证集合；一级标签数、Task Skill 数和可解实例数不得混为同一个数字。

9. 未决问题与后续工作

优先级	问题	下一步	完成条件
P0	has_water 仍是 Boolean	将状态定义、对象默认值、Rule、Process 和测试迁移到 [0,100]	水槽、花瓶、喷壶和咖啡机共享数值语义
P0	规划语义与 runtime 可能不一致	建立统一 Transition IR，供 runtime 和 PDDL compiler 共用	同一计划的预测 delta 与重放 delta 一致
P0	通用 schema grounding 缺失	实现 query、constraint、goal 的统一绑定器	多场景可批量生成 task instance
P1	Recipe 仍有专用代码分支	建立声明式 Recipe Registry 和通用 Process	新增产品只增加数据和必要能力
P1	时间过程的 PDDL 表达未定	在 PDDL2.1 durative action 与离散 tick compilation 中选定一种	洗衣、耗水和腐败可规划并可重放
P1	connect/rotate/consume 边界未定	用焊接、魔方和有限资源任务验证是否需要新 Action	每个新 Action 都对应不可替代的主体操作
P1	部分对象类型由自动规则误分类	校正对象模板的显式 family	容器、工具、设备、媒体等类型与语义一致
P1	六类标签尚未自动化	实现 goal-to-label 规则并运行现有任务语料	所有实例可重复得到相同多标签
P2	对象和状态表容易与代码漂移	从 registries 自动生成附录和覆盖统计	CI 检测文档快照与代码不一致

优先级	问题	下一步	完成条件
P2	文献任务映射仍是初步统计	扩展任务实例、目标谓词和未覆盖反例	能说明六类标签的覆盖范围和扩展点

附录 A：完整对象清单

下表以当前 130 个 `OBJECT_LIBRARY` 模板为基础。类型列采用目标语义类型，不保留当前自动推断产生的明显误分类；`has_water` 按目标设计写为数值默认值 0，其余状态沿用当前模板。对象表应在后续由代码自动生成。

Semantic Type	名称	类型	移动性	默认状态
<code>door</code>	门	结构物	不可移动	<code>is_open=False, is_dirty=False</code>
<code>button</code>	按钮	设备	不可移动	<code>is_on=False, is_pressed=False</code>
<code>room_light</code>	灯	照明设备	不可移动	<code>is_on=False</code>
<code>air_conditioner</code>	空调	设备	不可移动	<code>is_on=False, is_dirty=False</code>
<code>rack</code>	架子	家具	不可移动	<code>is_dirty=False</code>
<code>shoe_rack</code>	鞋架	家具	不可移动	<code>is_dirty=False</code>
<code>seat</code>	座椅	家具	不可移动	<code>is_dirty=False</code>
<code>chair</code>	椅子	家具	不可移动	<code>is_dirty=False</code>
<code>table</code>	桌子	家具	不可移动	<code>is_dirty=False</code>
<code>coffee_table</code>	茶几	家具	不可移动	<code>is_dirty=False</code>
<code>counter</code>	操作台	家具	不可移动	<code>is_dirty=False</code>
<code>desk</code>	书桌	家具	不可移动	<code>is_dirty=False</code>
<code>drawer</code>	抽屉	家具	不可移动	<code>is_open=False, is_dirty=False</code>
<code>sofa</code>	沙发	家具	不可移动	<code>is_dirty=False</code>
<code>bed</code>	床	家具	不可移动	<code>is_dirty=False</code>
<code>wardrobe</code>	衣柜	家具	不可移动	<code>is_open=False, is_dirty=False</code>
<code>cabinet</code>	柜子	家具	不可移动	<code>is_open=False, is_dirty=False</code>
<code>sink</code>	水槽	家具/容器	不可移动	<code>is_dirty=False, has_water=0</code>

Semantic Type	名称	类型	移动性	默认状态
faucet	水龙头	设备	不可移动	is_on=False
toilet	马桶	家具	不可移动	is_dirty=False
shower	淋浴	设备	不可移动	is_on=False,is_dirty=False
refrigerator	冰箱	设备	不可移动	is_open=False,is_dirty=False
microwave	微波炉	设备	不可移动	is_on=False,is_running=False,cycle_remaining=0, is_open=False,is_dirty=False
stove	炉灶	设备	不可移动	is_on=False,is_dirty=False
washing_machine	洗衣机	设备	不可移动	is_on=False,is_running=False,cycle_remaining=0, is_open=False,is_dirty=False
washer	洗衣机	设备	不可移动	is_on=False,is_running=False,cycle_remaining=0, is_open=False,is_dirty=False
drying_rack	晾衣架	家具	不可移动	无
television	电视	媒体设备	不可移动	is_on=False,is_dirty=False
display	显示屏	办公用品	不可移动	is_on=False,is_dirty=False
plant	植物	植物	可移动	is_wilted=False,is_wet=True,vitality=1.0
mug	杯子	容器	可移动	is_dirty=False,fill_level=0.0,is_full=False
cup	杯子	容器	可移动	is_dirty=False,fill_level=0.0,is_full=False, is_wet=False
plate	盘子	容器	可移动	capacity=1,is_dirty=False
bowl	碗	容器	可移动	is_dirty=False,capacity=1,is_wet=False
book	书	媒体物品	可移动	is_dirty=False
remote	遥控器	控制器	可移动	is_on=False,is_dirty=False,is_pressed=False
clothes	衣物	个人物品	可移动	folded=True,is_dirty=False,is_wet=False
shoes	鞋	个人物品	可移动	is_dirty=False,is_wet=False
box	箱子	容器	可移动	capacity=5,is_dirty=False
cart	推车	容器	可移动	capacity=5,is_dirty=False

Semantic Type	名称	类型	移动性	默认状态
computer	电脑	办公用品	不可移动	is_on=False, is_dirty=False
dishwasher	洗碗机	设备	不可移动	is_on=False, is_open=False, is_dirty=False
dispenser	分配器	设备	不可移动	fill_level=1.0, is_full=False, is_on=False, is_dirty=False
doctor_coat	医生白大褂	个人物品	可移动	is_dirty=False
drink	饮料	食品	可移动	is_open=False, is_rotten=False
fruit	水果	食品	可移动	is_rotten=False
hand_sanitizer_dispenser	免洗洗手液机	设备	不可移动	fill_level=1.0, is_full=False, is_on=False, is_dirty=False
juice	果汁	食品	可移动	is_open=False, is_rotten=False
knob	旋钮	设备	不可移动	is_on=False
locker	储物柜	办公用品	不可移动	is_open=False, is_dirty=False
machine	机器	设备	不可移动	is_on=False, is_dirty=False
medical_cart	医疗推车	医疗用品	可移动	is_dirty=False
medical_form	医疗表单	医疗用品	可移动	is_dirty=False
medicine_box	药盒	医疗用品	可移动	is_open=False
medicine_fridge	药品冰箱	设备	不可移动	is_open=False, is_dirty=False
milk	牛奶	食品	可移动	is_open=False, is_rotten=False
nurse_uniform	护士制服	个人物品	可移动	is_dirty=False
prescription_sheet	处方单	医疗用品	可移动	is_dirty=False
printer	打印机	设备	不可移动	is_on=False, is_running=False, cycle_remaining=0, is_dirty=False, count=0, amount=0
receipt	收据	办公用品	可移动	is_dirty=False
refrigerated_medicine	冷藏药品	食品	可移动	is_rotten=False, temperature=cold
shelf	货架	家具	不可移动	is_dirty=False

Semantic Type	名称	类型	移动性	默认状态
signboard	标牌	家具	不可移动	is_dirty=False
stationery	文具	办公用品	可移动	is_dirty=False
syringe	注射器	医疗用品	可移动	无
toilet_brush	马桶刷	清洁工具	可移动	is_dirty=False
toothbrush	牙刷	个人物品	可移动	is_dirty=False
toothpaste	牙膏	个人物品	可移动	is_dirty=False,uses_left=20
trash_bin	垃圾桶	容器	可移动	is_dirty=False
vegetable	蔬菜	食品	可移动	is_rotten=False
water_dispenser	饮水机	设备	不可移动	fill_level=1.0,is_full=False,is_on=True,is_dirty=False
wheelchair	轮椅	医疗用品	可移动	is_dirty=False
pillow	枕头	家具附件	可移动	is_dirty=False,is_wet=False
painting	画	装饰品	不可移动	is_dirty=False
vase	花瓶	容器	可移动	is_dirty=False,has_water=0
mirror	镜子	装饰品	不可移动	is_dirty=False
towel_holder	毛巾架	家具附件	不可移动	无
towel	毛巾	清洁工具	可移动	is_dirty=False,folded=True,is_wet=False
statue	雕像	装饰品	可移动	is_dirty=False
keychain	钥匙链	个人物品	可移动	无
cellphone	手机	媒体设备	可移动	is_dirty=False
bread	面包	食品	可移动	is_rotten=False,is_dirty=False
egg	鸡蛋	食品	可移动	is_rotten=False
fork	叉子	餐厨工具	可移动	is_dirty=False

Semantic Type	名称	类型	移动性	默认状态
spoon	勺子	餐厨工具	可移动	is_dirty=False
ladle	汤勺	餐厨工具	可移动	is_dirty=False
peppershaker	胡椒瓶	容器	可移动	is_dirty=False,uses_left=10
saltshaker	盐瓶	容器	可移动	is_dirty=False,uses_left=10
plunger	马桶吸	清洁工具	可移动	无
scrubbrush	清洁刷	清洁工具	可移动	is_dirty=False
soapbar	肥皂	清洁工具	可移动	is_dirty=False
tissuebox	纸巾盒	容器	可移动	is_dirty=False,count=100
dresser	梳妆柜	家具	不可移动	is_open=False,is_dirty=False
bathtub	浴缸	家具	不可移动	is_dirty=False
bathtubbasin	浴缸盆	容器	不可移动	is_dirty=False
newspaper	报纸	媒体物品	可移动	is_dirty=False
watch	手表	个人设备	可移动	is_dirty=False
tvstand	电视柜	家具	不可移动	is_dirty=False
teddybear	泰迪熊	装饰品	可移动	is_dirty=False
basketball	篮球	个人物品	可移动	无
tennisracket	网球拍	个人物品	可移动	无
baseballbat	棒球棒	个人物品	可移动	无
dumbbell	哑铃	个人物品	可移动	无
bottle	瓶子	容器	可移动	is_dirty=False
winebottle	酒瓶	容器	可移动	is_dirty=False
roomdecor	房间装饰	装饰品	可移动	is_dirty=False

Semantic Type	名称	类型	移动性	默认状态
poster	海报	装饰品	不可移动	is_dirty=False
ottoman	脚凳	家具	不可移动	is_dirty=False
footstool	脚踏凳	家具	不可移动	is_dirty=False
dogbed	宠物窝	家具	不可移动	is_dirty=False
garbagebag	垃圾袋	容器	可移动	无
aluminumfoil	铝箔纸	餐厨工具	可移动	无
tabletopdecor	桌面装饰	装饰品	可移动	is_dirty=False
vacuumcleaner	吸尘器	清洁设备	可移动	无
laundryhamper	洗衣篮	容器	可移动	is_open=False
coffeemachine	咖啡机	设备	不可移动	is_on=False, is_running=False, cycle_remaining=0, is_dirty=False
coffee	咖啡	食品	可移动	is_rotten=False, is_dirty=False
spraybottle	喷雾瓶	容器	可移动	has_water=0, uses_left=0
soapbottle	洗手液	容器	可移动	amount=10
clothesdryer	烘干机	设备	不可移动	is_on=False, is_running=False, cycle_remaining=0, is_open=False
cleaningcloth	抹布	清洁工具	可移动	is_dirty=False, is_wet=False
coffee_beans	咖啡豆	食品	可移动	无
tissue_refill	纸巾补充包	个人物品	可移动	无
soap_refill	洗手液补充装	个人物品	可移动	无
water_refill	水补充物	个人物品	可移动	无
pepper_refill	胡椒补充包	食品	可移动	无
salt_refill	盐补充包	食品	可移动	无
toothpaste_refill	牙膏补充装	个人物品	可移动	无

Semantic Type	名称	类型	移动性	默认状态
paper_pack	打印纸	办公用品	可移动	无
ink_cartridge	墨盒	办公用品	可移动	无

附录 B：文献与数据集映射

本表记录六类标签的初步来源和任务证据。它用于扩展任务语料，不表示六类已经构成完备或互斥的文献 taxonomy。详细来源保存在 [reference_table_graphworld_related_work.xlsx](#) 和 [task_planning_task_taxonomy.csv](#)。

来源	代表任务或机制	对应标签	对 GraphWorld 的作用
ALFRED	Clean & Place、Heat/Cool & Place、Pick & Place	clean、make、relocate	提供组合目标、导航、抓取、容器和设备操作实例
BEHAVIOR-1K	cleaning、personal care、cooking、物品整理	clean、make、relocate、operate	提供丰富状态谓词和长程家庭活动
VirtualHome	cooking、cleaning、laundry、Grab/Put/Open 程序	clean、make、relocate、operate	提供程序动作和状态更新表示
SayCan	取物、放置、递送、清理溢出物	clean、relocate、interact	提供语言技能选择和 affordance grounding
SayPlan	搜索物体、跨房间导航、整理和放置	relocate、navigate	提供场景图检索和长程子目标分解
MiniGrid / GridWorld	DoorKey、Unlock、Pickup、PutNext	operate、relocate、navigate	提供离散拓扑、钥匙门依赖和基础规划任务

后续统计以具体任务实例为单位，至少记录原始任务、初始条件、目标谓词、所需能力、代表动作、GraphWorld 标签和无法表达的部分。